Green University of Bangladesh
Department of Computer Science and Engineering(CSE)
Faculty of Sciences and Engineering

Project Report
Course Title:
Course Code:
Section:242-D3

Bangladesh Smart-Logistics Network Optimizer

Student Details

Name	ID
Masrur Hossain Seaim	242002123

Submission Date:
Course Teacher's Name:

[For Teachers use only: Don't Write Anything inside this box]

Report Status

Marks: …… **Signature:.....**
Comments:..... **Date:.....**

Contents

1. Introduction 3
1.1 Overview 3
1.2 Motivation 3
1.3 Problem Definition 3
1.3.1 Problem Statement 3
1.3.2 Complex Engineering Problem 3
1.4 Design Goals / Objectives 3
1.5 Application 4

2. Design / Development / Implementation 4
2.1 Project Details 4
2.2 Implementation 4
2.2.0 Pseudocode for Dijkstra 5
2.2.1 Flowchart 6
2.2.2 Output Screenshot 9

3. Performance Evaluation 10
3.1 Simulation Environment 10
3.2 Results and Discussion 10

4. Conclusion 10
4.1 Discussion 10
4.2 Limitations 10
4.3 Future Scope 10

Appendix: Core Algorithm (Dijkstra) 11

5. Source Code 12
5.1 Github Repository 18

Bangladesh Smart-Logistics Network Optimizer

1. Introduction

1.1 Overview

This project is a graph-based navigation and logistics optimization system designed to determine the shortest path between 10 major cities in Bangladesh. It uses Dijkstra's Algorithm to compute the most efficient routes and provides estimations for travel time and fuel cost.

1.2 Motivation

In modern supply chain systems, minimizing transportation cost and delivery time is essential. The motivation behind this project is to automate route planning for logistics services so that goods can be delivered in the most cost-effective and time-efficient way.

1.3 Problem Definition

1.3.1 Problem Statement

Manual route planning often results in higher fuel consumption and delayed deliveries. A computational system is required to analyze complex road networks and determine the optimal path.

1.3.2 Complex Engineering Problem

The road network is modeled as a Weighted Undirected Graph. Computing the shortest path in such a network while ensuring proper input validation introduces significant algorithmic complexity and logical challenges.

1.4 Design Goals / Objectives

- Implement Dijkstra's Algorithm to find the shortest path
- Provide travel time and fuel cost estimations based on vehicle type
- Ensure a professional command-line interface with strong input validation

1.5 Application

- **E-commerce Logistics:** Route optimization for delivery services like Daraj or Pathao
- **Supply Chain Management:** Efficient goods distribution from warehouses to hubs
- **Emergency Services:** Fast route planning for ambulances and fire services

2. Design / Development / Implementation

2.1 Project Details

The system is developed using the C programming language. An Adjacency Matrix is used to represent distances between cities.

Data Structures Used:

- Structures (struct) for city information
- Arrays for distance and path tracking

2.2 Implementation

The system is divided into three main components:

Graph Representation

- Nodes represent cities
- Edges represent distances in kilometers

Pathfinding Logic

- The algorithm selects the unvisited node with the smallest distance
- Updates distances of adjacent nodes iteratively

Estimation Logic

After finding the shortest path, the following formula is used:

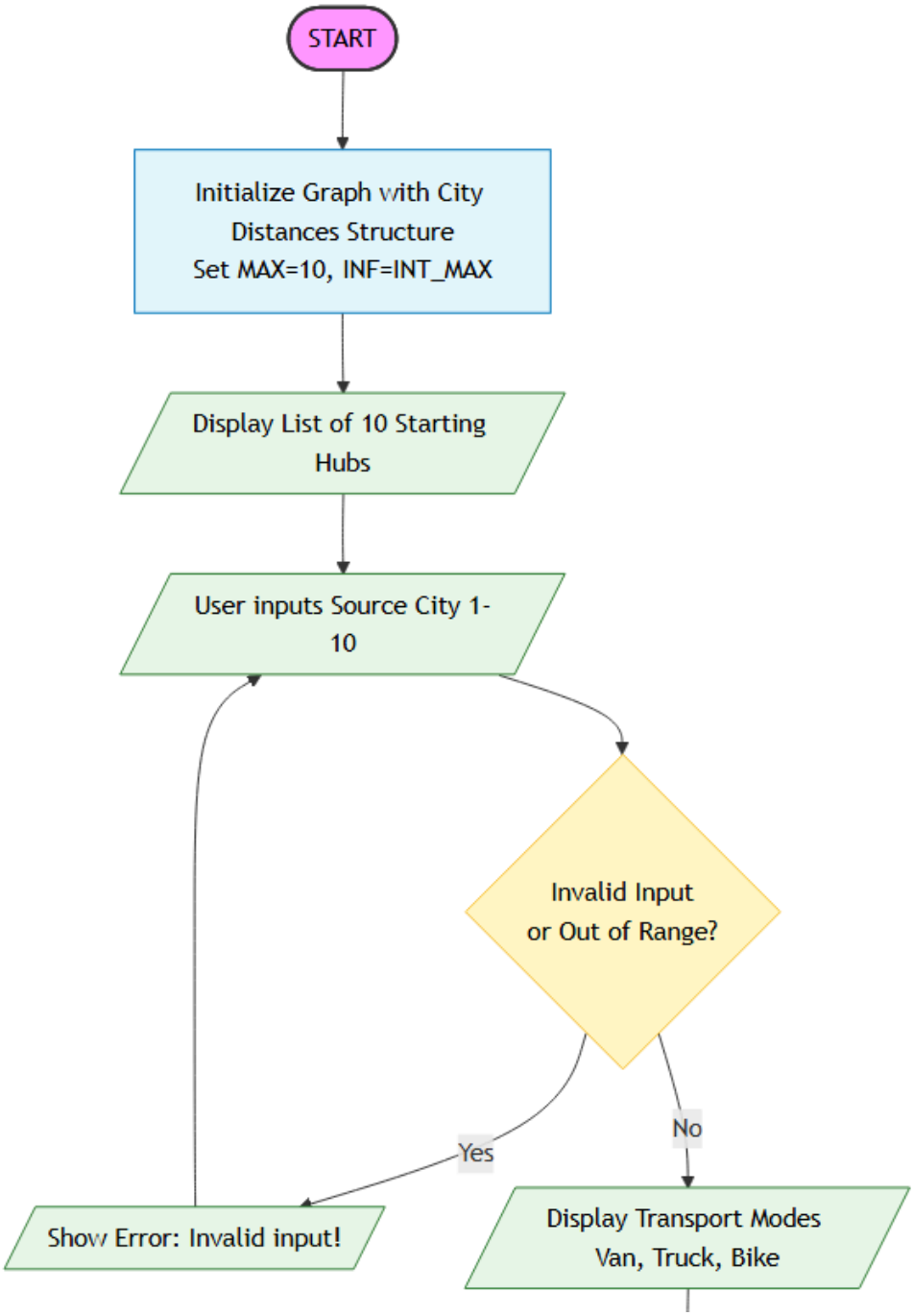
Time=Distance/Speed

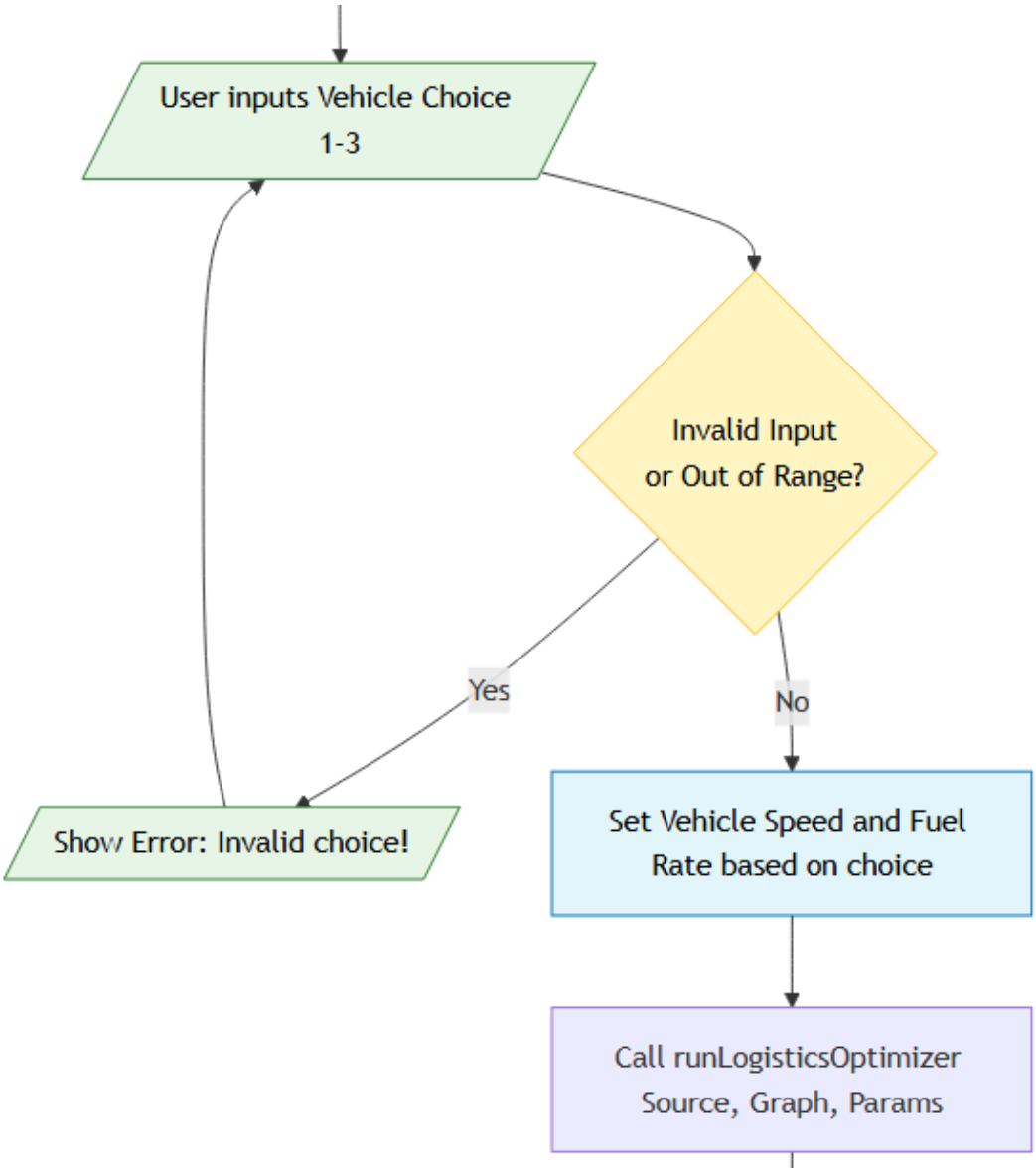
- Travel time is calculated
- Fuel cost is estimated based on vehicle type

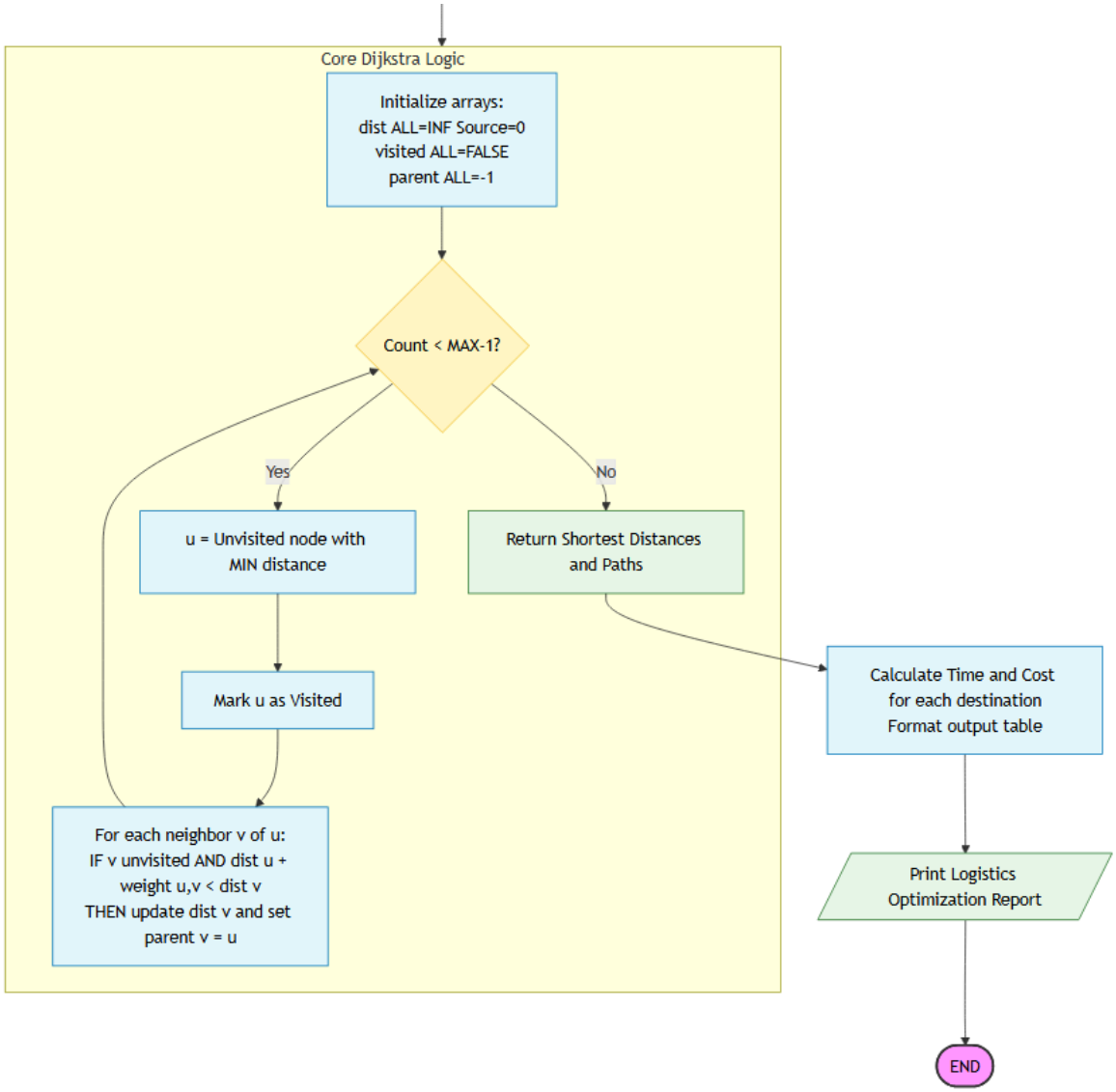
2.2.0 Pseudocode for Dijkstra

```
FUNCTION Dijkstra(Graph, Source):
    FOR each vertex v in Graph:
        dist[v] := INFINITY
        visited[v] := FALSE
        parent[v] := -1

    dist[Source] := 0

    FOR count FROM 0 TO Total_Vertices - 1:
        u := Vertex with Minimum dist[] and visited[u] == FALSE
        visited[u] := TRUE

        FOR each neighbor v of u:
            IF visited[v] == FALSE AND (dist[u] + weight(u, v) < dist[v]):
                dist[v] := dist[u] + weight(u, v)
                parent[v] := u

    RETURN dist, parent
```

Flowchart


START
Initialize Graph with City Distances Structure
Set MAX=10, INF=INT_MAX
Display List of 10 Starting Hubs
User inputs Source City 1-10
Invalid Input or Out of Range?
Yes
No
Show Error: Invalid input!
Display Transport Modes Van, Truck, Bike


User inputs Vehicle Choice
1-3
Invalid Input
or Out of Range?
Yes
No
Show Error: Invalid choice!
Set Vehicle Speed and Fuel
Rate based on choice
Call runLogisticsOptimizer
Source, Graph, Params


Core Dijkstra Logic
Initialize arrays:
dist ALL=INF Source=0
visited ALL=FALSE
parent ALL=-1
Count < MAX-1?
Yes
No
u = Unvisited node with
MIN distance
Return Shortest Distances
and Paths
Mark u as Visited
Calculate Time and Cost
for each destination
Format output table
For each neighbor v of u:
IF v unvisited AND dist u +
weight u,v < dist v
THEN update dist v and set
parent v = u
Print Logistics
Optimization Report
END

2.2.1 Output Screenshot

```
=====================================================================
     BANGLADESH SMART-LOGISTICS NETWORK OPTIMIZER (v3.2)
=====================================================================
Select Starting Distribution Hub:
[1] Dhaka        [2] Chittagong
[3] Sylhet       [4] Rajshahi
[5] Khulna       [6] Barishal
[7] Rangpur      [8] Mymensingh
[9] Cumilla      [10] Gazipur

Enter Option (1-10):
```

```
Enter Option (1-10): 5

Select Transport Mode for Cost & Time Estimation:
1. Delivery Van (Speed: 40km/h, Fuel: 12 TK/km)
2. Heavy Truck  (Speed: 30km/h, Fuel: 25 TK/km)
3. Express Bike (Speed: 60km/h, Fuel: 5 TK/km)
Enter Choice (1-3):
```

```
Enter Choice (1-3): 3

>>> Processing Optimal Routes from Khulna...

Destination     | Dist.    | Est. Time  | Fuel Cost    | Optimized Route
-----------------------------------------------------------------------------------------------
Dhaka           | 220 km  | 3.67   hrs | 1100.00 TK  | Khulna -> Dhaka
Chittagong      | 468 km  | 7.80   hrs | 2340.00 TK  | Khulna -> Dhaka -> Chittagong
Sylhet          | 460 km  | 7.67   hrs | 2300.00 TK  | Khulna -> Dhaka -> Sylhet
Rajshahi        | 470 km  | 7.83   hrs | 2350.00 TK  | Khulna -> Dhaka -> Rajshahi
Barishal        | 120 km  | 2.00   hrs | 600.00  TK  | Khulna -> Barishal
Rangpur         | 520 km  | 8.67   hrs | 2600.00 TK  | Khulna -> Dhaka -> Rangpur
Mymensingh      | 340 km  | 5.67   hrs | 1700.00 TK  | Khulna -> Dhaka -> Mymensingh
Cumilla         | 320 km  | 5.33   hrs | 1600.00 TK  | Khulna -> Dhaka -> Cumilla
Gazipur         | 245 km  | 4.08   hrs | 1225.00 TK  | Khulna -> Dhaka -> Gazipur

=====================================================================
Optimization finished.
```

3. Performance Evaluation

3.1 Simulation Environment

- Compiler: GCC (GNU Compiler Collection)
- IDE: Code::Blocks / Dev-C++ / VS Code

Simulation Procedure

- User selects a starting city (1–10)
- Selects a vehicle type
- System validates input
- Algorithm executes and displays results

3.2 Results and Discussion

- The system successfully computes shortest paths
- Correctly identifies intermediate cities
    - Example: Dhaka → Cumilla → Chittagong
- Input validation prevents system errors
- Execution time is very fast (within milliseconds)

4. Conclusion

4.1 Discussion

This project demonstrates the practical application of a greedy algorithm in real-world logistics. It effectively bridges theoretical concepts with functional implementation.

4.2 Limitations

- Does not consider real-time traffic conditions
- Limited to a command-line interface (no GUI)

4.3 Future Scope

- Integration of real-time traffic data
- Dynamic edge weight adjustment

- Development of a graphical user interface (GUI) using OpenGL or GTK

Appendix: Core Algorithm (Dijkstra)

Steps

1. **Initialization**
    - Source distance = 0
    - Other nodes = ∞
    - All nodes unvisited
2. **Update Distances**
    - For each neighbor, calculate:

$$NewDistance = Distance(current) + Weight$$

- Update if smaller

3. **Mark Visited**
    - Current node becomes visited
4. **Repeat**
    - Select the nearest unvisited node
    - Continue until all nodes are processed

Relaxation Formula

$$\text{if } dist[u] + w < dist[v] \Rightarrow dist[v] = dist[u] + w$$

Complexity

- Time:
  $O(V^2)$ using matrix
  $O((V + E) \log V)$ using heap
- Space:
  $O(V)$

5 Source Code

```
  #include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <limits.h>

#include <stdbool.h>


#define MAX 10

#define INF INT_MAX

typedef struct {

   char name[30];

} City;


int getMinDistance(int dist[], bool visited[], int n) {

   int min = INF, min_index = -1;

   for (int v = 0; v < n; v++) {

      if (!visited[v] && dist[v] <= min) {

         min = dist[v];

         min_index = v;

      }

   }

   return min_index;

}
```

```
void printPath(int parent[], int j, City cities[]) {
    if (parent[j] == -1) return;
    printPath(parent, parent[j], cities);
    printf(" -> %s", cities[j].name);
}
void runLogisticsOptimizer(int graph[MAX][MAX], int src, int n, City cities[], float fuelRate, int
speed) {
    int dist[MAX];
    bool visited[MAX];
    int parent[MAX];

    for (int i = 0; i < n; i++) {
        dist[i] = INF;
        visited[i] = false;
        parent[i] = -1;
    }

    dist[src] = 0;
    for (int count = 0; count < n - 1; count++) {
        int u = getMinDistance(dist, visited, n);
        if (u == -1) break;
        visited[u] = true;

        for (int v = 0; v < n; v++) {
```

```
            if (!visited[v] && graph[u][v] && dist[u] != INF
                && dist[u] + graph[u][v] < dist[v]) {
                parent[v] = u;
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
    printf("\n%-15s | %-8s | %-10s | %-12s | %-30s\n", "Destination", "Dist.", "Est. Time", "Fuel
Cost", "Optimized Route");

printf("-----------------------------------------------------------------------------------------
-----------\n");


    for (int i = 0; i < n; i++) {
        if (i == src) continue;


        if (dist[i] == INF) {
            printf("%-15s | %-8s | %-10s | %-12s | %-30s\n", cities[i].name, "N/A", "N/A", "N/A", "No
Route Found");
        } else {
            float time = (float)dist[i] / speed;
            float cost = (float)dist[i] * fuelRate;


            printf("%-15s | %-3d km  | %-6.2f hrs | %-7.2f TK  | %s",
                   cities[i].name, dist[i], time, cost, cities[src].name);
```

```
            printPath(parent, i, cities);
            printf("\n");
        }
    }
}
int main() {
    City cities[MAX] = {
        {"Dhaka"}, {"Chittagong"}, {"Sylhet"}, {"Rajshahi"}, {"Khulna"},
        {"Barishal"}, {"Rangpur"}, {"Mymensingh"}, {"Cumilla"}, {"Gazipur"}
    };
    int graph[MAX][MAX] = {
        {0, 248, 240, 250, 220, 180, 300, 120, 100, 25},
        {248, 0, 0, 0, 0, 0, 0, 0, 150, 0},
        {240, 0, 0, 0, 0, 0, 0, 0, 160, 0},
        {250, 0, 0, 0, 0, 0, 110, 0, 0, 0},
        {220, 0, 0, 0, 0, 120, 0, 0, 0, 0},
        {180, 0, 0, 0, 120, 0, 0, 0, 0, 0},
        {300, 0, 0, 110, 0, 0, 0, 200, 0, 0},
        {120, 0, 0, 0, 0, 0, 200, 0, 0, 100},
        {100, 150, 160, 0, 0, 0, 0, 0, 0, 0},
        {25, 0, 0, 0, 0, 0, 0, 100, 0, 0}
    };
    int sourceChoice, vehicleChoice;
    float fuelRate = 0;
```

```
    int speed = 0;

printf("==================================================================
=\n");

    printf("     BANGLADESH SMART-LOGISTICS NETWORK OPTIMIZER (v3.2)           \n");

printf("==================================================================
=\n");


    printf("Select Starting Distribution Hub:\n");

    for (int i = 0; i < MAX; i++) {

        printf("[%d] %-12s ", i + 1, cities[i].name);

        if ((i + 1) % 2 == 0) printf("\n");

    }

    while (1) {

        printf("\nEnter Option (1-10): ");

        if (scanf("%d", &sourceChoice) != 1) {

            printf("Error: Invalid input! Please enter a valid NUMBER (1-10).\n");

            while(getchar() != '\n');

            continue;

        }

        if (sourceChoice >= 1 && sourceChoice <= MAX) {

            break;

        } else {

            printf("Error: Out of range! Please enter a city number between 1 and 10.\n");
```

```
    }
}
printf("\nSelect Transport Mode for Cost & Time Estimation:\n");
printf("1. Delivery Van (Speed: 40km/h, Fuel: 12 TK/km)\n");
printf("2. Heavy Truck  (Speed: 30km/h, Fuel: 25 TK/km)\n");
printf("3. Express Bike (Speed: 60km/h, Fuel: 5 TK/km)\n");

while (1) {
    printf("Enter Choice (1-3): ");
    if (scanf("%d", &vehicleChoice) != 1) {
        printf("Error: Invalid input! Please enter 1, 2, or 3.\n");
        while(getchar() != '\n');
        continue;
    }
    if (vehicleChoice >= 1 && vehicleChoice <= 3) {
        break;
    } else {
        printf("Error: Invalid choice! Choose between 1, 2, or 3.\n");
    }
}
if (vehicleChoice == 1) { speed = 40; fuelRate = 12.0; }
else if (vehicleChoice == 2) { speed = 30; fuelRate = 25.0; }
else { speed = 60; fuelRate = 5.0; }
```

```
    printf("\n>>> Processing Optimal Routes from %s...\n", cities[sourceChoice - 1].name);

    runLogisticsOptimizer(graph, sourceChoice - 1, MAX, cities, fuelRate, speed);


printf("\n==================================================================
===\n");

    printf("Optimization finished.\n");

    return 0;

}
```

5.1 Github Repository link for source code:

[Masrur-Siam/Bangladesh-Smart-Logistics-Network-Optimizer-ALGORITHM-project]